Temporizadores de hardware

Un **temporizador** cuenta pulsos de reloj por su cuenta; cuando el contador llega a la **alarma**, dispara una **interrupción** y la CPU ejecuta una **ISR** corta que solo levanta una **bandera**.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Del reloj a la ISR

la CPU solo configura, arranca y detiene; el timer cuenta solo

1 — Reloj APB · 80 MHz `RELOJ`
APB = bus de periféricos. 80 000 000 pulsos por segundo: cada uno dura **12,5 ns**.

2 — Prescaler ÷ N `DIVISOR`
Deja pasar **1 de cada N** pulsos. Con N = 80: 1 000 000 de ticks por segundo.

```
f = 80 000 000 / N
```

3 — Contador `REGISTRO + SUMADOR`
Suma **1 por tick**, sin que nadie se lo pida. Con N = 80, 1 tick = **1 µs**.

4 — ¿contador = alarma?
El comparador revisa en cada tick.

Sí → **Interrupción → ISR**
La CPU pausa `loop()` y corre la ISR.

No → No: el contador sigue contando (paso 3)

2 ¿Qué me dan? → qué fórmula

? ¿Qué dato tengo?

tiempo y f
Busco la alarma.

```
alarma = tiempo × f
```

alarma y f
Busco cada cuánto avisa.

```
tiempo = alarma / f
```

solo f
Busco prescaler y duración del tick.

```
N = 80 000 000 / f
```

LAS FÓRMULAS (¡DE MEMORIA!)

FRECUENCIA DE TICK `f = 80 000 000 / N`	PRESCALER `N = 80 000 000 / f`	RESOLUCIÓN (UN TICK) `resolución = 1 / f`
TIEMPO HASTA EL AVISO `tiempo = alarma / f`	ALARMA PARA UN TIEMPO `alarma = tiempo × f`	AVISOS POR SEGUNDO `f / alarma`

EJEMPLOS RESUELTOS

1 · UN AVISO CADA MEDIO SEGUNDO

- f = 1 000 000, tiempo = 0,5 s
- Resolución = 1 / 1 000 000 s = **1 µs**
- Alarma = 0,5 × 1 000 000 = **500 000**
- Comprobación: 500 000 × 1 µs = 0,5 s
- Prescaler: N = 80 000 000 / 1 000 000 = **80**

2 · OTRA FRECUENCIA DE TICK

- f = 10 000, alarma = 2 500
- Resolución = 1 / 10 000 s = **100 µs**
- Tiempo = 2 500 × 100 µs = **0,25 s** (4 avisos por segundo)
- Prescaler: N = 80 000 000 / 10 000 = **8 000** (válido)

F (TICKS/S)	RESOLUCIÓN
1 000 000 (1 MHz)	1 µs
10 000 (10 kHz)	100 µs

Regla de la clase: ticks de 1 µs (1 MHz): la alarma se escribe directamente en µs.

N	TICKS	EFECTO
Pequeño	Cortos	Más precisión; se llena antes
Grande	Largos	Cubre más tiempo; menos precisión

3 Autorrecarga: el aviso se repite

el contador vuelve solo a 0

1 — El contador cuenta
0, 1, 2, 3…

2 — ¿Llegó a la alarma?
El comparador revisa en cada tick.

No → **Sigue contando**
Vuelve al paso 1.

Sí

3 — Aviso
Se genera el evento: interrupción.

4 — Vuelve a 0
Autorrecarga (*auto-reload*): sin que el programa haga nada.

vuelve al paso 1: avisos a intervalos iguales

	ALARMA + AUTORRECARGA	DESBORDAMIENTO
Lo provoca	Un valor que **tú eliges**	El **máximo** que cabe en los bits
Qué pasa	Aviso y vuelve a 0	Da la vuelta a 0
¿Intencional?	Sí: uso normal	Casi nunca: límite físico

CONTADOR	MÁXIMO	SE LLENA EN…
8 bits (AVR)	255	0,26 ms
16 bits (AVR)	65 535	65,5 ms
64 bits (ESP32)	$2^{64} - 1$	≈ 584 000 años

ESP32 VS. ATMEGA328P: TEMPORIZADORES

	ESP32 (WROOM-32)	ATMEGA328P (UNO)
Temporizadores de uso general	**4** (TIMG0 y TIMG1), en paralelo	3 (Timer0, Timer1, Timer2)
Tamaño del contador	64 bits	8 o 16 bits
Reloj de entrada	80 MHz	16 MHz
Prescaler	Cualquier N de 2 a 65 535	Solo 1, 8, 64, 256 o 1024
Se llena (sin prescaler)	Prácticamente nunca	Timer1: en 4,1 ms

LAS 4 PIEZAS DE LA ISR

1 · CALLBACK

Función que tú escribes pero **no llamas**: se la entregas al timer para que la llame cuando ocurra algo.

Imaginalo: «llámeme cuando el auto esté listo»: tu número es el callback.

2 · IRAM_ATTR

Etiqueta antes del nombre de la ISR: pide copiarla a la **IRAM** (RAM interna rápida), no dejarla en la flash (lenta y a veces ocupada).

Imaginalo: el extintor en la pared, no en el sótano con llave.

3 · VOLATILE

Obliga a leer la variable **de la memoria cada vez**. Sin ella, el compilador (que no ve la ISR) guarda una copia y `loop()` nunca ve el cambio.

Regla: si la comparten la ISR y `loop()`, es `volatile`.

4 · PATRÓN BANDERA

La ISR solo pone `bandera = true` y termina; `loop()` revisa, la baja y hace el trabajo lento.

Imaginalo: el buzón rural: el cartero levanta la banderita; el dueño la baja y lee con calma.

COMPLEMENTO · CORE 3.X (NO ESTÁ EN LA DIAPOSITIVA)

```
hw_timer_t *timer = nullptr;
volatile bool tick = false;
void IRAM_ATTR onTimer() { tick = true; }      // ISR: solo la bandera

void setup() {
  timer = timerBegin(1000000);                  // f = 1 MHz → 1 tick = 1 µs
  timerAttachInterrupt(timer, &onTimer);        // registra el callback
  timerAlarm(timer, 500000, true, 0);           // alarma 500 000 µs, autorrecarga
}
```

En la 2.x: `timerBegin(0, 80, true)`, `timerAlarmWrite(...)`, `timerAlarmEnable(...)`: no compila en la 3.x.

RESPUESTAS · ACTIVIDAD EN PAREJAS

01 *A · f = 1 000 000 y alarma = 2 000. ¿Cada cuánto ocurre el aviso?*
tiempo = 2 000 / 1 000 000 = 0,002 s = **2 ms** (500 avisos por segundo).

02 *B · Quiero un aviso cada 10 ms con f = 1 000 000. ¿Qué alarma uso?*
alarma = 0,010 × 1 000 000 = **10 000**.

03 *C · f = 40 000 y alarma = 40 000. ¿Cada cuánto? ¿Qué prescaler?*
tiempo = 40 000 / 40 000 = **1 s**. N = 80 000 000 / 40 000 = **2 000**.